



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
آزمایشگاه معماری کامپیوتر
دکتر صفری
گزارش نهایی

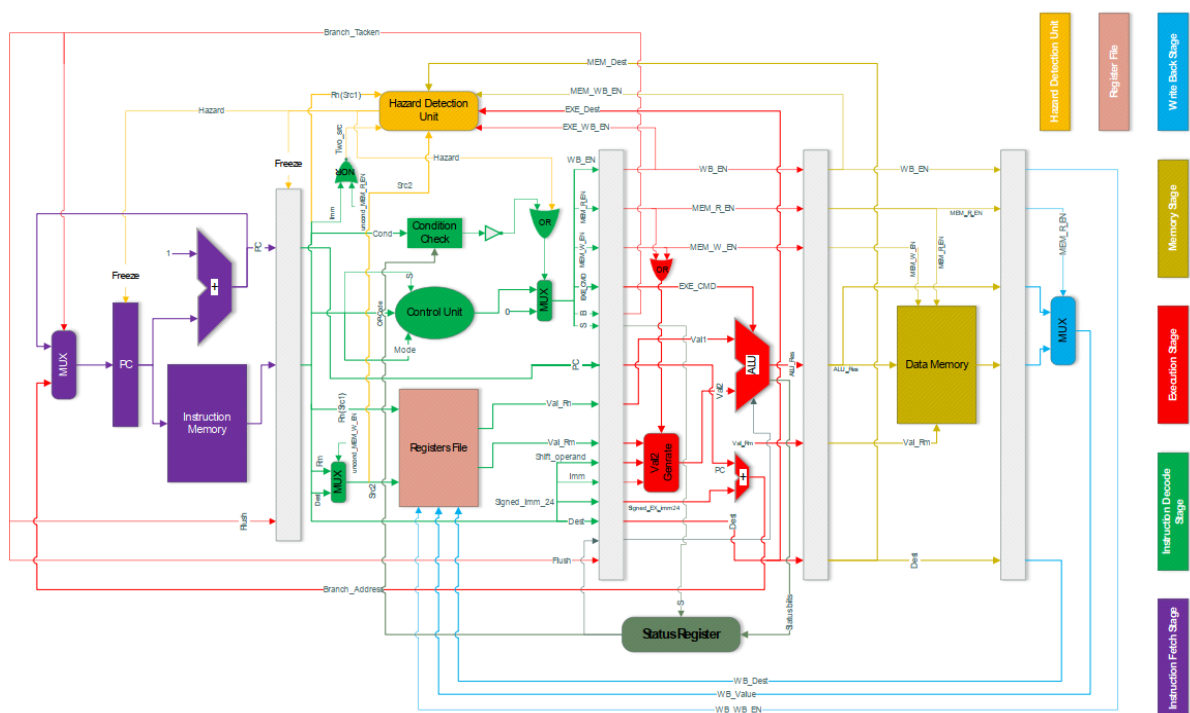
نام و نام خانوادگی	علیرضا نجفی مطیعی -
شماره دانشجویی	۸۱۰۱۰۰۲۲۴ -
تاریخ ارسال گزارش	۱۴۰۴.۱۱.۱۰

فهرست

۳	۱. کلیات پردازنده ARM
۴	2. IF Stage
۷	3. ID Stage
۱۰	Status Register
۱۱	Control Unit
۱۲	Condition Check
۱۳	Register File
۱۵	4. EXE Stage
۱۶	Val2 Generate
۱۶	ALU
۱۸	EXE Adder
۲۱	5. MEM Stage
۲۱	Distributed Memory Generator
۲۲	6. WB Stage
۲۲	7. Hazard Unit
۲۵	8. Forwarding Unit
۲۷	9. SRAM
۳۲	10. گزارش عملکرد
۳۵	11. مشکلات هنگام انجام پروژه

۱. کلیات پردازنده ARM

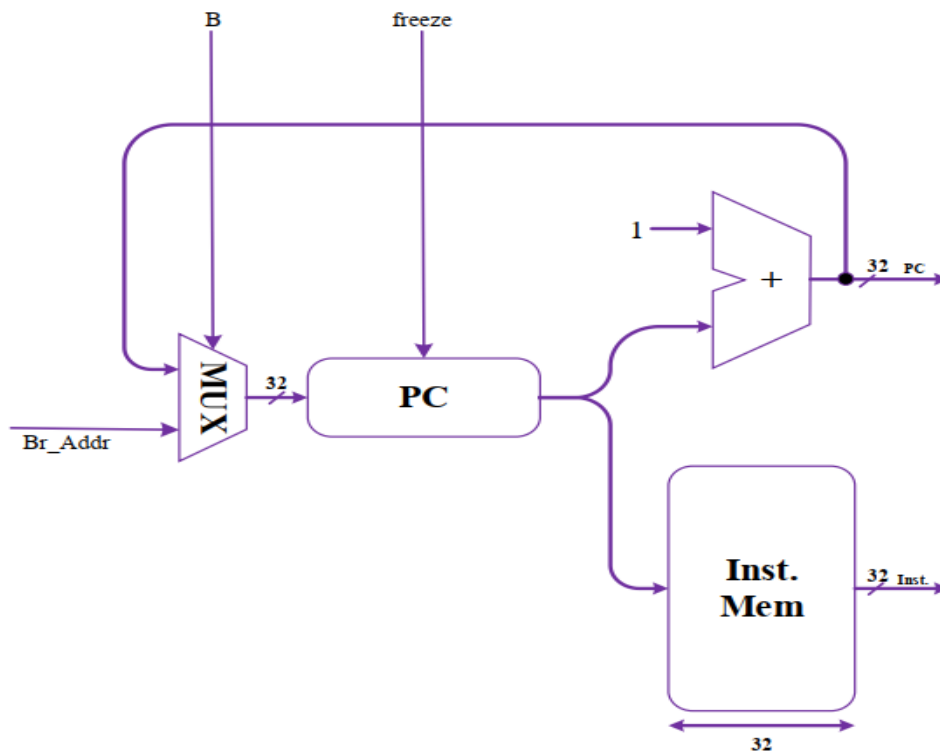
در این آزمایشگاه معماری پردازنده ARM968E-S که از خانواده ARM9 و ARM9E می‌باشد در دستور کار قرار گرفته‌است که به صورت مرحله به مرحله و سپس افزودن هر بخش به یکدیگر و همچنین افزودن قابلیت‌هایی مانند Hazard Detection Unit و Forwarding و یا جایگزینی SRAM می‌باشد. در شکل ۱ شمای کلی این پردازنده را می‌توان مشاهده کرد که شامل ۵ بخش اصلی IF, ID, EXE, MEM, WB است. همچنین از دیگر نکات قابل ذکر راجع به این پردازنده این است که آدرس‌دهی بر حسب کلمه و فضای آدرس دستورات و داده تفکیک شده می‌باشد. در ادامه هر مازول را بیشتر بررسی خواهیم کرد.



شکل ۱. شمای کلی پردازنده ARM

IF Stage .2

این بخش واحد واکشی دستور است که شماتیک کلی این واحد در شکل ۲ مشخص شده است. وظیفه این واحد خواندن (واکشی) دستورات عمل‌ها از حافظه، یکی پس از دیگری، برای ارسال به مرحله بعد است.



شکل ۲. شمای کلی IF Stage

درواقع در این بخش با استفاده از یک رجیستر (PC) Program Counter که به صورت دنباله‌وار افزایش می‌یابد یا در دستور branch دچار پرش می‌شود دستورات را واکشی می‌کنیم.

همانطور که مشخص است این ماژول شامل ورودی‌های مانند Br_Addr, rst, clk و سیگنال‌های کنترلی Branch_Taken و Freeze است. همچنین خروجی‌های این ماژول مقدار آپدیت شده PC (PC دستور بعد) و همچنین آدرس حافظه برای بازخوانی دستور هستند. در این ماژول یک مولتیپلکسر داریم که بین آدرس پی‌سی بعدی یا آدرس برنج هدف با توجه به برنج بودن یا نبودن دستور انتخاب می‌کند. همچنین مقدار PC در هر سیکل یکی افزایش می‌یابد مگر اینکه ریست یا سیگنال فریز فعال شده باشند که در این صورت ثابت می‌ماند. برای Inst.Mem از Distributed Memory Generator IP موجود در ویوادو استفاده می‌کنیم که ۸۱۹۲ خانه ۳۲ بیتی دارد و همچنین فایل دستورات را به آن ورودی داده‌ایم. خروجی آن یک دستور ۳۲ بیتی است با استفاده از خروجی رجیستر PC انتخاب می‌شود. همچنین یک ادر داریم که برای افزایش مقدار PC به اندازه یک عدد استفاده می‌شود.

کد ماژول کلی به همراه زیربخش‌های این ماژول که در بالا توضیح داده شد در زیر آورده شده‌است:

```
module IF ( clk , rst , freeze , BranchAddr , Branch_taken , PC , PC_out, Inst );

input clk , rst , freeze , Branch_taken ;
input [31:0] BranchAddr;
output [31:0] PC , PC_out , Inst ;
wire [31:0] Inst_temp;
wire [10:0] PC_out_temp = PC_out[10:0];
wire [31:0] Inst =Inst_temp ;

wire [31:0] PC_in , adderIn;
assign adderIn =32'd1;
PC pc(.PC_in(PC_in) , .PC(PC_out) , .clk(clk) , .rst(rst) , .freeze(freeze));
Adder adder1( .inA(PC_out) , .inB(adderIn) , .out(PC));
MUX2to1 mux1(.inA(PC) , .inB(BranchAddr) , .sel(Branch_taken) , .out(PC_in));

dist_mem_gen_1 Mem_inst (
    .a(PC_out_temp),          // input wire [10 : 0] a
    .spo(Inst_temp)           // output wire [31 : 0] spo
);

endmodule
```

```
module PC (PC_in , PC , clk , rst , freeze);

input clk ,rst , freeze;
input [31:0] PC_in ;
output reg [31:0] PC;

always@ (posedge clk , posedge rst) begin

    if (rst)
        PC <= 32'd0;
    else if (freeze)
        PC <= PC ;
    else
        PC <=PC_in ;

end

endmodule
```

```
module MUX2to1 (inA , inB , sel , out );
input [31:0] inA , inB ;
output [31:0] out;
input sel;

assign out = (sel) ? inB :inA;
endmodule
```

```

module Adder ( inA , inB , out);

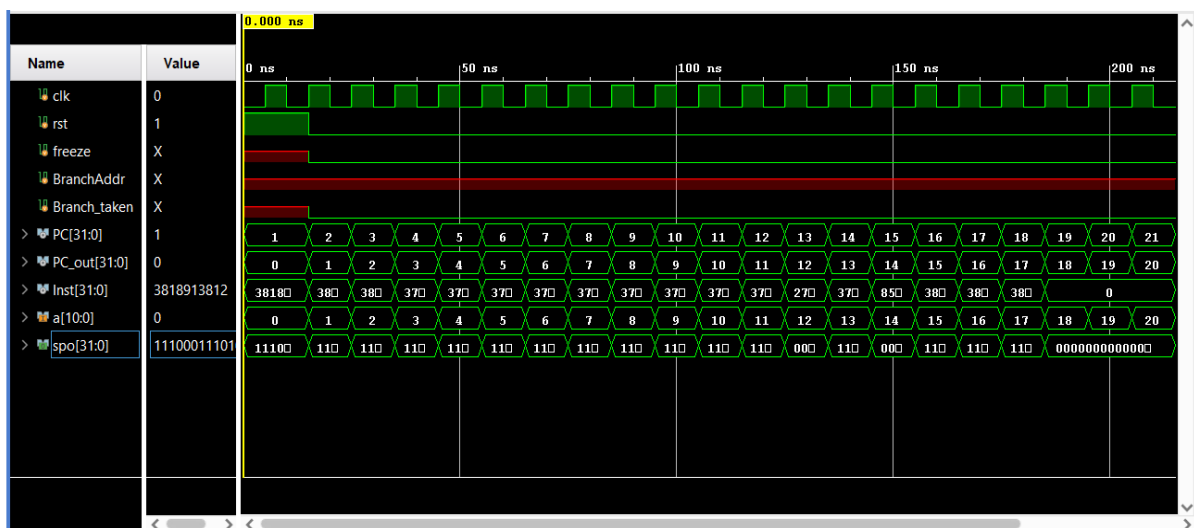
input [31:0] inA , inB ;
output [31:0] out ;
assign out = inA +inB;

endmodule

```

همچنین بین این ماژول و ماژول بعدی یک رجیستر قرار دارد که مانند یک بافر عمل می کند. در طراحی این پردازنده در بین هر طبقه یک رجیستر قرار دارد که تا انتهای گزارشکار یک نمونه از آن قرار می گیرد.

خروجی شبیه سازی این بخش نیز در زیر آورده شده است:



شکل ۳. خروجی شبیه سازی IF Stage

همانطور که مشاهده می شود خروجی مورد نظر برای این ماژول این بود که PC به درستی افزایش پیدا کند و دستورات نیز به درستی وارد شوند که این اتفاق به درستی افتاده است و نتایج مورد قبولند.

3. ID Stage

پیش از اینکه به توضیح خود این ماژول بپردازیم نیاز است تا کمی بیشتر به نحوه کار این پردازنده از نظر دستوری بپردازیم.

R-type Instructions		Description	Bits							
			31:28	27:26	25	24:21	20	19:16	15:12	11:00
			Cond.	Mode	I	OP-Code	S	Rn	Rd	shifter operand
0	NOP ^{۱۱}	No Operation	1110	00	0	0000	0	0000	0000	0000000000
1	MOV	Move	cond	00	I	1101	S	0000	Rd	shifter operand
2	MVN ^{۱۲}	Move NOT	cond	00	I	1111	S	0000	Rd	shifter operand
3	ADD	Add	cond	00	I	0100	S	Rn	Rd	shifter operand
4	ADC	Add with Carry	cond	00	I	0101	S	Rn	Rd	shifter operand
5	SUB	Subtraction	cond	00	I	0010	S	Rn	Rd	shifter operand
6	SBC	Subtract with Carry	cond	00	I	0110	S	Rn	Rd	shifter operand
7	AND	And	cond	00	I	0000	S	Rn	Rd	shifter operand
8	ORR	Or	cond	00	I	1100	S	Rn	Rd	shifter operand
9	EOR	Exclusive OR	cond	00	I	0001	S	Rn	Rd	shifter operand
10	CMP	Compare	cond	00	I	1010	1	Rn	0000	shifter operand
11	TST ^{۱۳}	Test	cond	00	I	1000	1	Rn	0000	shifter operand
12	LDR	Load Register	cond	01	0	0100	1	Rn	Rd	offset_12
13	STR	Store Register	cond	01	0	0100	0	Rn	Rd	offset_12
14	B	Branch	cond	10	1	0	signed_immed_24			

جدول ۱. لیست دستورات پردازنده

با شروع از سمت چپ جدول ستون اول و دوم یعنی R-type Instructions و Description می‌باشند که تایپ یا نوع دستور را مشخص می‌کنند. ستون بعدی که Cond است نشان می‌دهد که در پردازنده ARM دستورات به صورت شرطی انجام می‌شوند و این ۴ بیت است که تعیین می‌کند دستور به صورت شرطی قرار است اجرا شود یا خیر. در جدول شماره ۲ لیست حالت‌های اجرای دستورات ذکر شده‌است. در صورتیکه یک دستور به صورت غیر شرطی اجرا شود مقدار بیت‌های شرط برابر ۱۱۱۰ خواهد بود و در صورتیکه شرط برقرار نباشد دستور مانند NOP هیچ کاری انجام نخواهد داد. در بخش Mode مد دستور مشخص شده‌است. دستوراتی که مد آن‌ها ۰۰ است دستورات محاسباتی هستند. دستوراتی که مد ۰۱ دارند از نوع دستورات کار

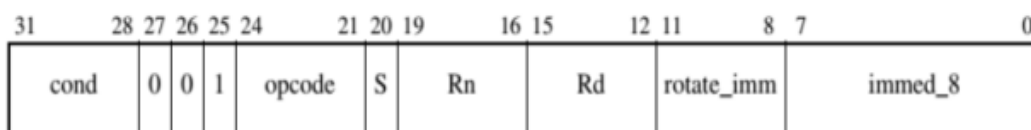
با حافظه هستند و مد دستور پرش نیز ۱۰ است. بیت بیست و پنجم آدرس یعنی I، Immediate بودن دستور را نشان می‌دهد که آیا اینگونه است یا با رجیستر کار می‌کند (اگر یک باشد Immediate است و اگر صفر باشد کار با رجیستر دارد). بیت‌های ۲۱ تا ۲۴ مربوط به Opcode می‌باشد که کد دستورات عمل برای تعیین نوع دستور است. بیت ۲۰ یعنی S تعیین می‌کند که Status Register باید آپدیت شود یا خیر. مثلاً دستورات TST و CMP بیت S برابر با یک دارند بنابراین نیاز است تا Status Register آپدیت شود چرا که این دستورات را قبل از شرط اجرا می‌کنیم و برای اجرای دستورات شرطی لازم است تا Status Register آپدیت شود. همچنین Rd رجیستر مقصد را نشان می‌دهد و نتیجه عملیات باید در آن رجیستر ذخیره شود. Rn نیز همان سورس اول می‌باشد و به عنوان یکی از عملوندهای دستورات مورد استفاده قرار می‌گیرد.

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
0000	EQ	Equal	Z set
0001	NE	Not equal	Z clear
0010	CS/HS	Carry set/unsigned higher or same	C set
0011	CC/LO	Carry clear/unsigned lower	C clear
0100	MI	Minus/negative	N set
0101	PL	Plus/positive or zero	N clear
0110	VS	Overflow	V set
0111	VC	No overflow	V clear
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N set and V set, or N clear and V clear (N == V)
1011	LT	Signed less than	N set and V clear, or N clear and V set (N != V)
1100	GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0, N == V)
1101	LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	Always (unconditional)	-
1111	-	See Condition code 0b1111	-

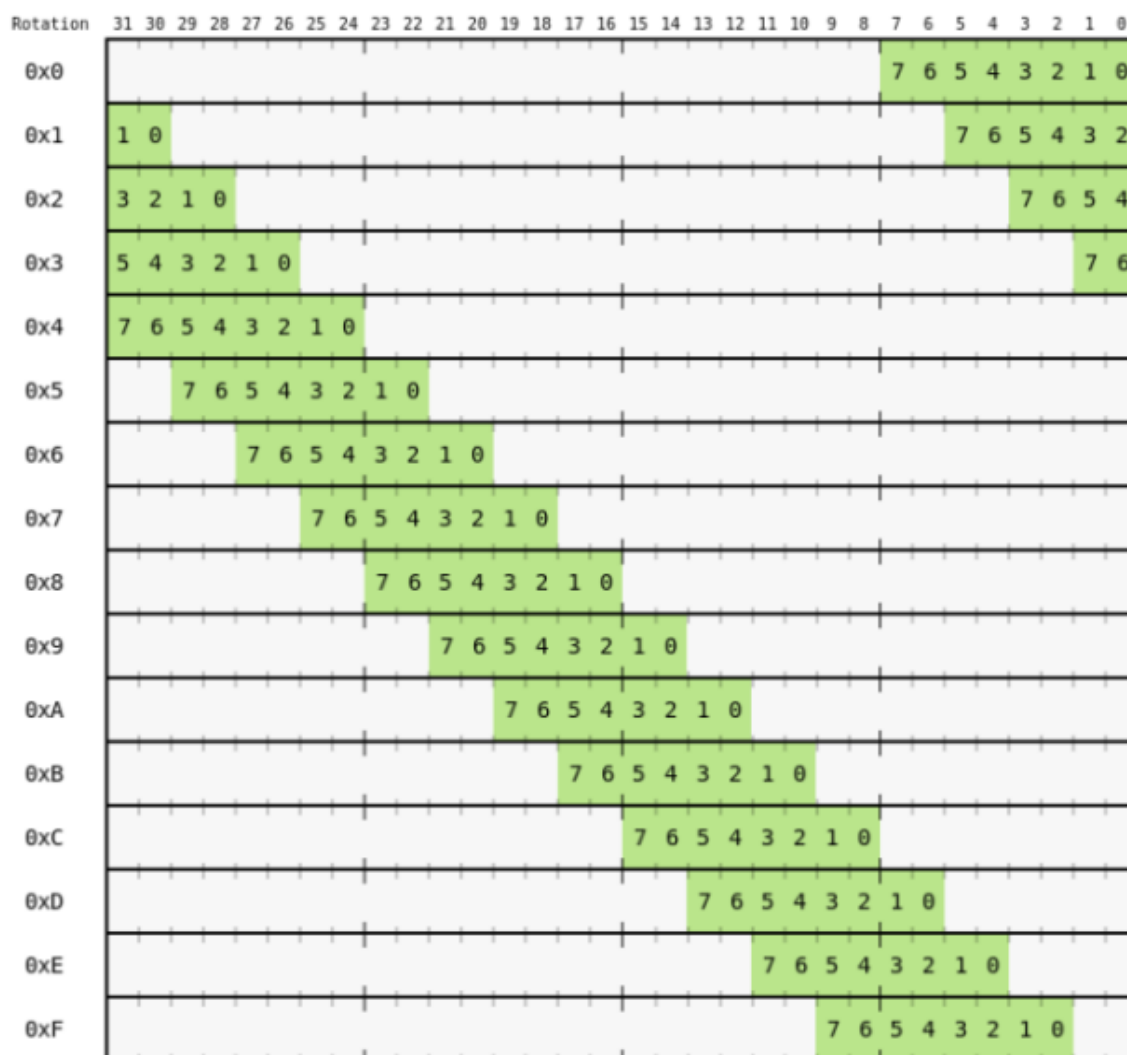
جدول ۲. کد شرط دستورات

قسمت آخر جدول یعنی بیت‌های ۱۱ تا ۰ بیت‌های shifter operand هستند که حالات مختلفی دارند.

۱- 32-bit immediate: در این حالت مقدار بیت I برابر با یک است. عدد ۸ بیتی imm8- در یک بخش ۳۲ بیت قرار می‌گیرد سپس به اندازه دو برابر imm_rotate به راست روتیت می‌شوند.



شکل ۴. دستور العمل از نوع ۳۲ بیت فوری



شکل ۵. نحوه چرخش عدد فوری

در این پردازنده ۴ بیت پرارزش shift operand را به عنوان rotate_imm در نظر گرفته شده‌است تا با چرخش بخش ۸ بیتی به میزان دو برابر این مقدار چرخش انجام بگیرد.

۲- Immediate Shift: در این حالت بیت I و بیت چهارم دستورالعمل برابر با صفر است. همچنین عملوند دوم از رجیستر خوانده می‌شود و سپس عدد خوانده شده بر اساس حالت شیفت به مقدار imm_shift شیفت داده می‌شود. حالت‌های شیفت در جدول ۳ آورده شده‌است.

مقدار	توضیحات	وضعیت شیفت
00	Logical shift left	LSL
01	Logical shift right	LSR
10	Arithmetic shift right	ASR
11	Rotate right	ROR

جدول ۳. وضعیت شیفت در دستورات شیفت فوری

31	28	27	26	25	24	21	20	19	16	15	12	11	7	6	5	4	3	0
cond	0	0	0	opcode	S	Rn	Rd	shift_imm	shift	0	Rm							

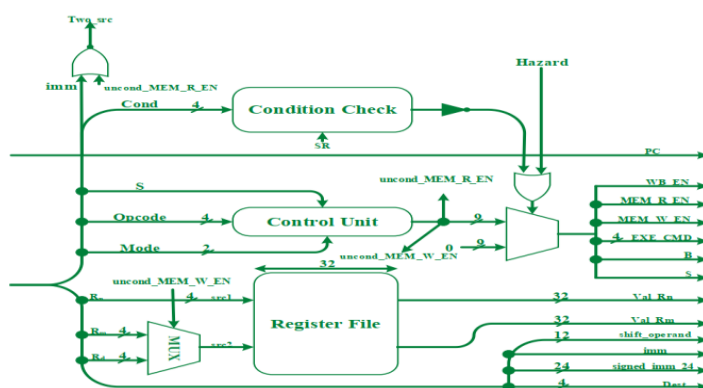
شکل ۶. دستورالعمل از نوع شیفت فوری

چهار بیت Rm را باید مطابق با جدول ۳ شیفت دهیم و بیت‌های ۴ و ۵ مشخص می‌کنند که شیفت باید به چه صورتی انجام شود. بیت‌های ۷ تا ۱۱ (shift-imm) نیز مشخص می‌کند که چه مقدار شیفت باید انجام شود.

Status Register

همچنین در پردازنده ARM یک رجیستر به منظور نگهداری وضعیت کلی پردازنده لحاظ شده‌است. درواقع این رجیستر بیان می‌کند که mode و وضعیت اجرای دستورات در پردازنده به چه صورت است و برای بررسی شرط مورد استفاده قرار می‌گیرد. ۴ بیت Z, N, C و V در یک رجیستر به نام Status Register قرار می‌گیرند و نوشتن در این رجیستر با لبه پایین رونده کلاک انجام می‌شود. همچنین اینکه رجیستر باید آپدیت شود یا خیر با استفاده از بیت S مشخص می‌شود.

بخش ID درواقع دستورات خام را به یکسری دستور قابل اجرا تبدیل می‌کند. بخش اصلی این قسمت به نوعی بخش Control Unit است که درواقع سیگنال‌های پایپ‌لاین را برای عملیات‌های پردازنده تولید می‌کند. شمای کلی این بخش در زیر آورده شده‌است:



شکل ۷. شمای کلی ID Stage

Control Unit

این بخش opcode دستورات را دیکود می‌کند و یکسری سیگنال کنترلی برای پایپ‌لاین تولید می‌کند. درواقع ۴ بیت opcode، ۲ بیت mode و بیت S را دریافت می‌کند و ۹ بیت کنترل خروجی می‌دهد. خروجی شامل کدهای عملیاتی ALU، کنترل‌های مربوط به دسترسی به حافظه و انیبل برای Write Back می‌باشد. در واقع یک Case Statement نوع‌های مختلف دستور را به سیگنال‌های کنترلی آن‌ها که پردازش داده، حافظه و دستورات پرش را هندل می‌کنند مرتبط می‌سازد. در دستوراتی مانند LDR و STR که Opcode و Mode دو دستور یکسان است از بیت S به منظور تمایز استفاده می‌شود و در خروجی مثلاً MEM_W_EN برای STR یک می‌شود و MEM_R_EN برای LDR یک می‌شود. WB_EN نیز باید برای همه دستوراتی که قرار است در رجیستر فایل چیزی نوشته شود یک شود.

```
module controller (cond ,mode, opcode , S_IN , WB_EN , MEM_R_EN , MEM_W_EN , EXE_CMD , B ,S_OUT );
input [3:0] opcode;
input [1:0] mode;
input [3:0] cond;
input S_IN;
output reg WB_EN, MEM_R_EN, MEM_W_EN, B, S_OUT;
output reg [3:0] EXE_CMD;

always @(opcode , mode , S_IN)begin
{MEM_R_EN, MEM_W_EN, B, EXE_CMD} = 7'd0;
S_OUT = S_IN;
WB_EN = 1'b0;
case (mode)
2'b00: begin
case (opcode)
4'b1101: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0001;
B = 1'b0;
end
4'b1111: begin
WB_EN = 1'b1;
EXE_CMD = 4'b1001;
B = 1'b0;
end
4'b0100: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0010;
end
4'b0101: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0011;
end
4'b0010: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0100;
end
end
4'b0110: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0101;
B = 1'b0;
end
end
4'b0000: begin
if (cond!= 4'b1110)begin
WB_EN = 1'b0;
end
else begin
WB_EN = 1'b1;
EXE_CMD = 4'b0110;
B = 1'b0;
end
end
end
4'b1100: begin
WB_EN = 1'b1;
EXE_CMD = 4'b0111;
B = 1'b0;
end
end
4'b0001: begin
WB_EN = 1'b1;
EXE_CMD = 4'b1000;
B = 1'b0;
end
end
end
```

```

4'b1010 :begin
    WB_EN = 1'b0;
    EXE_CMD = 4'b0100;
    B = 1'b0;
end
4'b1000 :begin
    WB_EN = 1'b0;
    EXE_CMD = 4'b0110;
    B = 1'b0;
end
endcase
end
2'b01: begin
    if (opcode == 4'b0100)begin
        if (S_IN==1) begin
            WB_EN = 1'b1;
            EXE_CMD = 4'b0010;
            MEM_R_EN = 1'b1;
            B = 1'b0;
        end
        else begin
            WB_EN = 1'b0;
            EXE_CMD = 4'b0010;
            MEM_W_EN = 1'b1;
            B = 1'b0;
        end
    end
end
2'b10: begin
    B = 1'b1;
end
endcase
end
endmodule

```

Condition Check

این ماژول بررسی می‌کند که آیا یک دستور با توجه به ۴ بیت شرط و ۴ بیت Status register قرار است اجرا شود یا خیر. اگر شرط برقرار باشد خروجی یک خواهد شد و دستور باید اجرا شود و در غیر این صورت باید با آن مانند یک NOP رفتار شود. با توجه به مالتی پلکسری که در خروجی بخش کنترل وجود دارد و نحوه اتصال ورودی‌ها، سلکت آن باید نقیض خروجی Condition Check باشد. در صورت وجود هازارد خروجی‌ها نباید به رجیستر بروند برای همین یک گیت OR در نظر گرفته شده‌است.

```

module Condition_Check(
    input [3:0] cond,
    input [3:0] status, // [N Z C V] = [3:0]
    output reg out
);

wire Z = status[3];
wire C = status[2];
wire N = status[1];
wire V = status[0];

always @(*) begin
    case (cond)
        4'b0000: out = Z; // EQ: Z set
        4'b0001: out = ~Z; // NE: Z clear
        4'b0010: out = C; // CS/HS: C set
        4'b0011: out = ~C; // CC/LO: C clear
        4'b0100: out = N; // MI: N set
        4'b0101: out = ~N; // PL: N clear
        4'b0110: out = V; // VS: V set
        4'b0111: out = ~V; // VC: V clear
        4'b1000: out = C & ~Z; // HI: C set and Z clear
        4'b1001: out = ~C | Z; // LS: C clear or Z set
        4'b1010: out = (N == V); // GE: N == V
        4'b1011: out = (N != V); // LT: N != V
        4'b1100: out = ~Z & (N == V); // GT: Z clear and N == V
        4'b1101: out = Z | (N != V); // LE: Z set or N != V
        4'b1110: out = 1'b1; // AL: Always
        4'b1111: out = 1'b0; // Reserved (Never execute)
        default: out = 1'b0; // Default safe
    endcase
end
endmodule

```

Register File

این بخش شامل ۱۶ رجیستر ۳۲ بیتی است که به عنوان حافظه اولیه داده پردازنده کار می‌کند. نوشتن در آن روی لبه پایین رونده کلاک انجام می‌شود تا از هازارد نوشتن پس از خواندن جلوگیری شود. این بخش دو پورت خواندن و یک پورت نوشتن دارد. ورودی اول آن همان Rn است و ورودی دوم آن خروجی MUX است که وجود این MUX به این دلیل است که در همه دستورات به جز دستور STR ورودی دوم از shifter operand می‌آید و در دستور STR مقدار Rd به عبارتی مقدار سورس است. درواقع همواره Rm را به عنوان ورودی دوم می‌خوانیم و در همه دستورات غیر از STR ورودی دوم از shifter operand می‌آید اما اگر دستور ما STR باشد اینگونه نخواهد بود و Rd به عنوان ورودی دوم قرار خواهد گرفت. به همین دلیل است که MUX در ورودی دوم RF قرار داده شده است و سلکت آن را نیز MEM_W_EN قرار دادیم تا اگر دستور STR باشد ورودی دوم MUX به ورودی دوم RF برود. ابتدا در هر رجیستر ۰ تا ۱۵ به ترتیب مقادیر ۰ تا ۱۵ ریخته می‌شود.

```
module Register_File ( clk , rst , Src1 , Src2 , Dest_WB , Result_WB , Write_Back_EN , Reg1 , Reg2 );

input clk , rst , Write_Back_EN;
input [3:0] Src1 , Src2 , Dest_WB;
input [31:0] Result_WB;
output [31:0] Reg1 ,Reg2;

reg [31:0] RF [15:0];

//read
assign Reg1 = RF[Src1];
assign Reg2 = RF[Src2];

integer i;

//write
always @(negedge clk , posedge rst)
begin
    if (rst) begin
        RF[0] <= 32'd0;
        for (i = 0; i < 16; i = i + 1)
            RF[i] <= i ;
        end
    else if(Write_Back_EN)begin
        RF[Dest_WB] <= Result_WB;
    end
end

endmodule
```

کد کلی ماژول نیز در زیر آمده است:

```
module ID_stage(clk, rst, Instruction, Result_WB, Write_Back_EN, Dest_WB, Hazard, SR, WB_EN, MEM_R_EN,
MEM_W_EN, B, S, EXE_CMD, Val_Rn, Val_Rm, Imm, Shift_Operand, Signed_Imm_24, Dest, Src1, Src2, Two_src
);

input clk, rst, Write_Back_EN, Hazard;
input [31:0] Instruction, Result_WB;
input [3:0] Dest_WB, SR;

output WB_EN, MEM_R_EN, MEM_W_EN, B, S, Two_src, Imm;
output [3:0] EXE_CMD, Dest, Src1, Src2;
output [31:0] Val_Rn, Val_Rm;
output [11:0] Shift_Operand;
output [31:0] Signed_Imm_24;

wire CC_OUT, Sel_Mux2to1_9;
wire [3:0] Mux4_OUT;

wire [8:0] CT_OUT, Mux9_OUT;
wire CT_WB_EN, CT_MEM_R_EN, CT_MEM_W_EN, CT_B, CT_S;
wire [3:0] CT_EXE_CMD;
assign CT_OUT = {CT_WB_EN, CT_MEM_R_EN, CT_MEM_W_EN, CT_B, CT_S, CT_EXE_CMD};

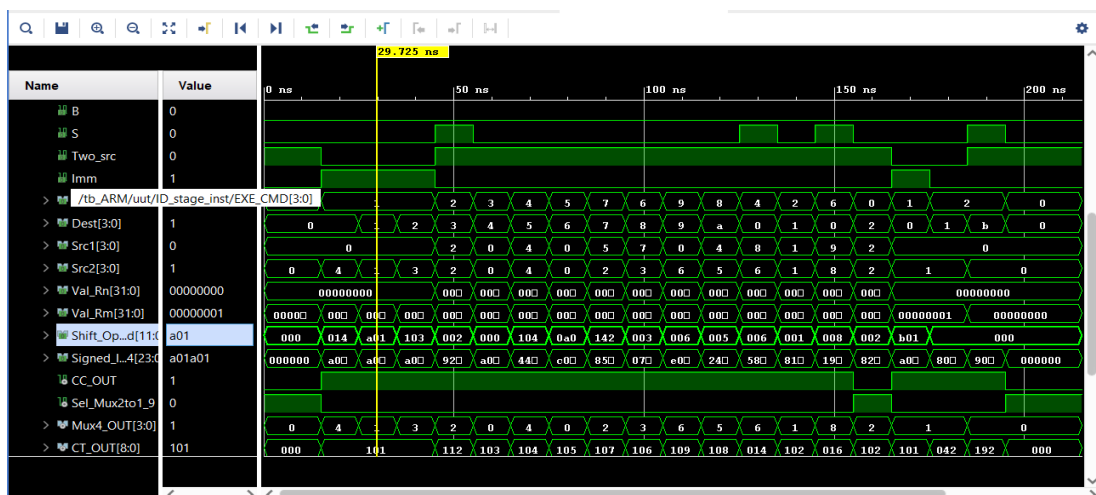
NOR NOR1(Instruction[25], MEM_W_EN, Two_src);
OR OR2(~CC_OUT, Hazard, Sel_Mux2to1_9);
Condition_Check CC(.cond(Instruction[31:28]), .status(SR), .out(CC_OUT));
Mux2to1_4 M1(.a(Instruction[3:0]), .b(Instruction[15:12]), .sel(MEM_W_EN), .out(Mux4_OUT));
Mux2to1_9 M2(.a(CT_OUT), .b(9'b00000000), .sel(Sel_Mux2to1_9), .out(Mux9_OUT));
Register_File RF(.clk(clk), .rst(rst), .Src1(Instruction[19:16]), .Src2(Mux4_OUT), .Dest_WB(Dest_WB), .Result_WB(Result_WB), .Write_Back_EN(Write_Back_EN), .Reg1(Val_Rn), .Reg2(Val_Rm));

controller CT(.cond(Instruction[31:28]), .mode(Instruction[27:26]), .opcode(Instruction[24:21]), .S_IN(Instruction[20]), .WB_EN(CT_WB_EN),
.MEM_R_EN(CT_MEM_R_EN), .MEM_W_EN(CT_MEM_W_EN), .EXE_CMD(CT_EXE_CMD), .B(CT_B), .S_OUT(CT_S));

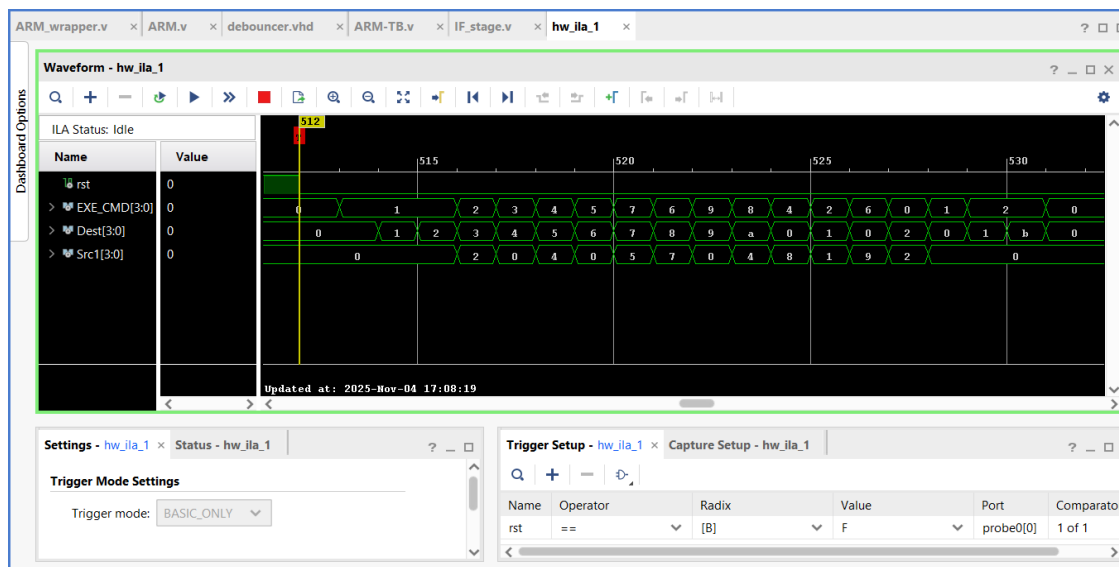
assign WB_EN = Mux9_OUT[8];
assign MEM_R_EN = Mux9_OUT[7];
assign MEM_W_EN = Mux9_OUT[6];
assign B = Mux9_OUT[5];
assign S = Mux9_OUT[4];
assign EXE_CMD = Mux9_OUT[3:0];
assign Signed_Imm_24 = {{8{Instruction[23]}}, Instruction[23:0]};
assign Shift_Operand = Instruction[11:0];
assign Imm = Instruction[25];
assign Dest = Instruction[15:12];
assign Src1 = Instruction[19:16];
assign Src2 = Mux4_OUT;

endmodule
```

نتایج شبیه‌سازی این بخش نیز به صورت زیر است:



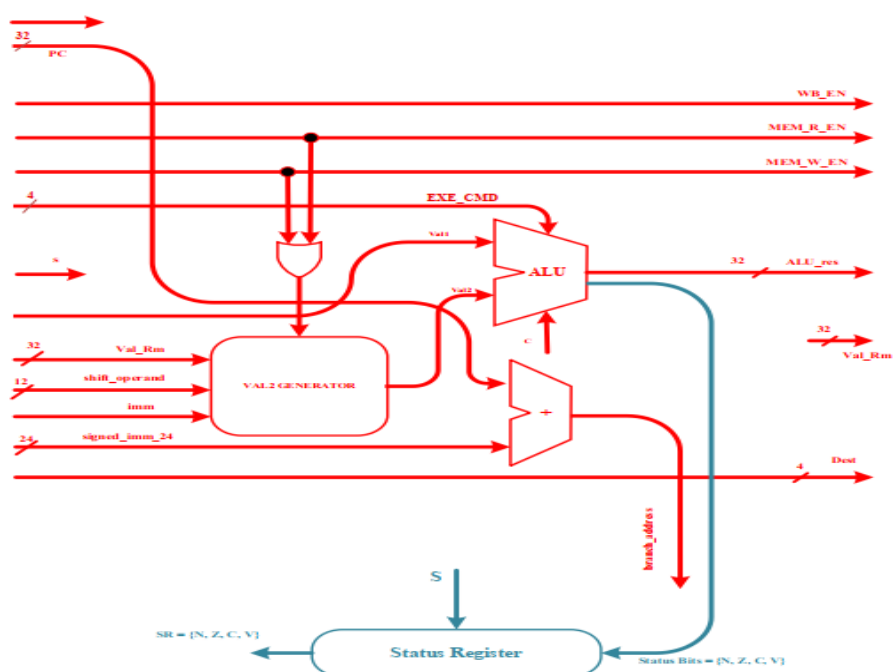
شکل ۸. شبیه‌سازی بخش ID



شکل ۹. نتیجه شبیه‌سازی این بخش روی FPGA

4. EXE Stage

در این بخش دستوراتی که در مراحل قبلی fetch و decode شده‌اند، اجرا می‌شوند. درواقع عملیات مشخص‌شده توسط دستور داده شده باید توسط CPU به انجام برسد. عملیات‌های منطقی و ریاضی توسط ALU، تولید مقدار دوم توسط val2gen و یک جمع‌کننده برای تولید آدرس پرش بخش‌های اصلی این مازول را تشکیل می‌دهند.



شکل ۱۰. شمای کلی EXE Stage

Val2 Generate

این ماژول ورودی دوم ALU را با توجه به دستور مشخص می‌کند. در واقع سه حالت مختلف را هندل می‌کند که به شرح زیر است:

۱. برای دستورات فوری ($I=1$) یک ۸ بیت فوری استخراج می‌کند و به صورت مشخص شده آن را روتیت می‌کند.

۲. برای دستورات حافظه ۱۲ بیت آفست را با صفر اسکتند می‌کند.

۳. برای shifted register operand یعنی $I=0$ و $Mem_in=0$ این ماژول شیفت منطقی لازم را با استفاده از shift_operand روی val_rm اعمال می‌کند.

```
module Val2_Generate (
    Val_Rm,
    Shift_operand,
    imm,
    Out_or,
    Val2
);
    input [31:0] Val_Rm;
    input [11:0] Shift_operand;
    input imm;
    input Out_or;
    output reg [31:0] Val2;

    wire [63:0] temp_rotate;
    assign temp_rotate = {24'b0, Shift_operand[7:0], 24'b0, Shift_operand[7:0]};
    wire [63:0] temp_Rm;
    assign temp_Rm = {Val_Rm, Val_Rm};

    always @(*) begin
        if (Out_or) Val2 = {20'b0, Shift_operand};
        else begin
            if (imm == 1) Val2 = temp_rotate[31+(Shift_operand[11:8]<<1)-:32];
            else begin
                case (Shift_operand[6:5])
                    2'b00: Val2 = Val_Rm << Shift_operand[11:7];
                    2'b01: Val2 = Val_Rm >> Shift_operand[11:7];
                    2'b10: Val2 = $signed(Val_Rm) >>> Shift_operand[11:7];
                    2'b11: Val2 = temp_Rm[31+(Shift_operand[11:7])-:32];
                endcase
            end
        end
    end
endmodule
```

ALU

این ماژول عملیات‌های منطقی و ریاضی را بر اساس ۴ بیت دستور انجام می‌دهد. ورودی اول آن همان val_Rn اتس و مستقیم از مرحله قبل می‌آید و val2 نیز خروجی Val2 Generate می‌باشد. این بخش دستوراتی مانند MOV, ADD, ADC, SUB, AND, ORR, EOR را پشتیبانی می‌کند. خروجی عملیات به همراه ۴ بیت فلگ وضعیت به عنوان خروجی این ماژول هستند. Z زمانی یک می‌شود که حاصل

صفر است، N زمانی یک می‌شود که نتیجه منفی باشد، C کری است و برای ADC و SBC استفاده می‌شود و V نیز برای اورفلو در جمع و تفریق است. عملیات‌های موجود با توجه به جدول ۴ توسط ALU به انجام می‌رسند.

Instruction	ALU Command	Operation
MOV	0001	result = in2
MVN	1001	result = $\sim$ in2
ADD	0010	result = in1 + in2
ADC	0011	result = in1 + in2 + C
SUB	0100	result = in1 - in2
SBC	0101	result = in1 - in2 - $\sim$ C
AND	0110	result = in1 & in2
ORR	0111	result = in1 in2
EOR	1000	result = in1 ^ in2
CMP	0100	result = in1 - in2
TST	0110	result = in1 & in2
LDR	0010	result = in1 + in2
STR	0010	result = in1 + in2
B	XXXX	

جدول ۴. ریزدستورهای ALU

```

module ALU( EXE_CMD, Val1, Val2, Cin, C,ALU_Res ,N ,Z, V );
input [3:0] EXE_CMD;
input [31:0] Val1, Val2;
input Cin;
output reg C;
output N;
output reg V;
output Z;

output reg [31:0]ALU_Res;

always @(Val1 , Val2 , EXE_CMD , Cin ) begin
    V = 1'b0;
    C = 1'b0;
    case (EXE_CMD)
        4'b0001:
            ALU_Res = Val2;
        4'b1001:
            ALU_Res = ~Val2;
        4'b0010: begin
            {C,ALU_Res} = Val1 + Val2;
            V = (~Val1[31] & ~Val2[31] & ALU_Res[31]) | (Val1[31] & Val2[31] & ~ALU_Res[31]);
        end
        4'b0011: begin
            {C, ALU_Res} = Val1 + Val2 + Cin;
            V = (~Val1[31] & ~Val2[31] & ALU_Res[31]) | (Val1[31] & Val2[31] & ~ALU_Res[31]);
        end
        4'b0100:begin
            {C, ALU_Res} = Val1 - Val2;
            V = (~Val1[31] & Val2[31] & ALU_Res[31]) | (Val1[31] & ~Val2[31] & ~ALU_Res[31]); // Subtract
        end
        4'b0101: begin
            {C, ALU_Res} = Val1 - Val2 - !Cin;
            V = (~Val1[31] & Val2[31] & ALU_Res[31]) | (Val1[31] & ~Val2[31] & ~ALU_Res[31]); // Subtract
        end
        4'b0110:
            ALU_Res = Val1 & Val2;
        4'b0111:
            ALU_Res = Val1 | Val2;
        4'b1000:
            ALU_Res = Val1 ^ Val2;
        default: begin
            ALU_Res = 32'b0;
        end
    endcase
end

assign Z = (ALU_Res == 32'd0) ?1:0;
assign N = ALU_Res[31];
endmodule

```

این جمع‌کننده برای محاسبه آدرس مقصد پرش استفاده می‌شود. این آدرس در زمانی مورد نیاز است که پردازنده دستوری از نوع Branch را بخواهد اجرا کند. این جمع‌کننده مقدار فعلی PC و مقدار اکستندد عدد فوری مربوط به دستور پرش (Signed Immediate) را دریافت می‌کند. ابتدا مقدار Immediate از ID وارد شده و به صورت علامت دار به ۳۲ بیت اکستندد می‌شود سپس این مقدار با مقدار PC جمع می‌شود. خروجی این جمع‌کننده آدرس مقصد پرش است که مستقیم به MUX انتخاب PC در IF متصل است. کد این بخش و همچنین کد بخش رجیستر وضعیت که در بخش قبل توضیح آن داده شد در اینجا آمده‌است:

```
module Adder ( inA , inB , out);

input [31:0] inA , inB ;
output [31:0] out ;
assign out = inA +inB;

endmodule
```

```
module Status_Reg( clk , rst , Status_bits_IN , Status_bits_OUT , S_IN );

input [3:0] Status_bits_IN ;
input S_IN , clk , rst ;
output reg [3:0] Status_bits_OUT;

always @ (negedge clk , posedge rst)begin
    if (rst) begin
        Status_bits_OUT <= 4'b0000;
    end
    else begin
        if(S_IN)
            Status_bits_OUT <= Status_bits_IN;
        end
    end
end
endmodule
```

همچنین کد کلی این طبقه در صفحه بعد آمده است:

```

module EXE_stage (
    clk,
    rst,
    EXE_CMD,
    MEM_R_EN,
    MEM_W_EN,
    PC,
    Val_Rn,
    Val_Rm,
    Imm,
    Shift_Operand,
    Signed_Imm_24,
    S,
    SR,
    ALU_Result,
    Br_Addr,
    Status
);

input clk, rst, MEM_R_EN, MEM_W_EN, Imm,S;
input [3:0] EXE_CMD, SR;
input [11:0] Shift_Operand;
input [31:0] Signed_Imm_24;
input [31:0] PC, Val_Rm, Val_Rn;

output [31:0] ALU_Result, Br_Addr;
output [3:0] Status;

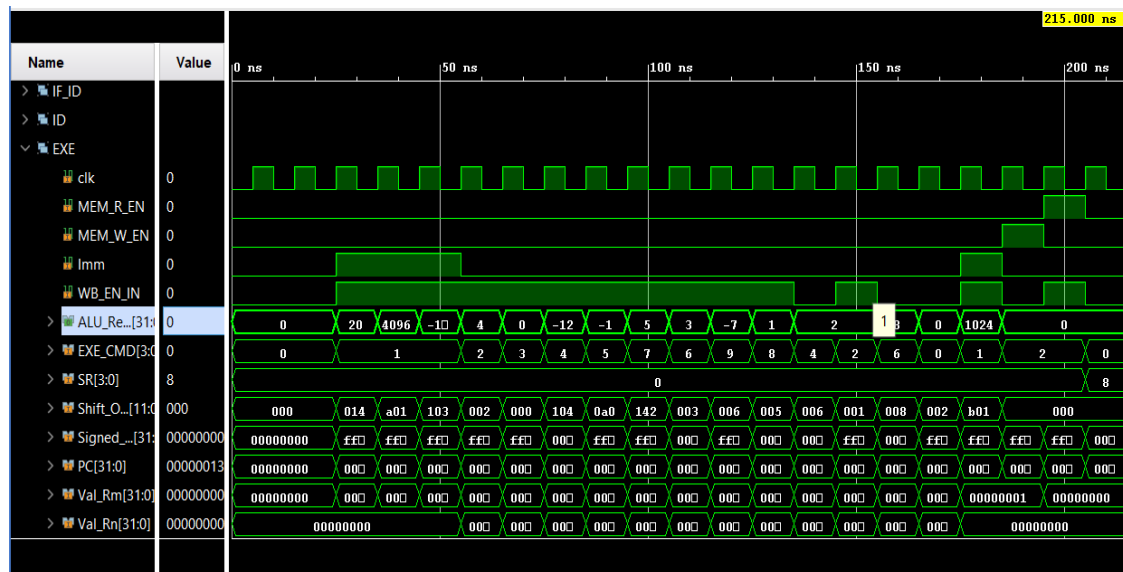
wire Out_or;
wire [31:0] Val2;
wire N, Z, C, V;
ALU A1 (
    .EXE_CMD(EXE_CMD),
    .Val1(Val_Rn),
    .Val2(Val2),
    .Cin(SR[1]),
    .C(C),
    .ALU_Res(ALU_Result),
    .N(N),
    .Z(Z),
    .V(V)
);
Val2_Generate V1 (
    .Val_Rm(Val_Rm),
    .Shift_operand(Shift_Operand),
    .imm(Imm),
    .Out_or(Out_or),
    .Val2(Val2)
);
Adder A2 (
    .inA(PC),
    .inB(Signed_Imm_24),
    .out(Br_Addr)
);
OR O1 (
    MEM_W_EN,
    MEM_R_EN,
    Out_or
);

wire[3:0] ss;
assign ss = {Z,C,N,V};
Status_Reg Status_Reg_inst( clk , rst , {Z,C,N,V} , Status , S );

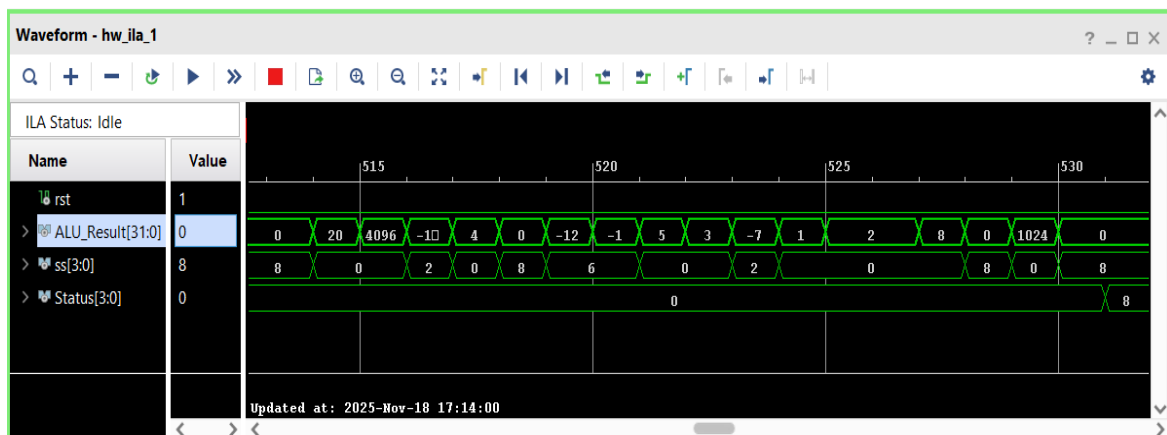
endmodule

```

همچنین نتایج شبیه‌سازی این بخش به صورت زیر است:



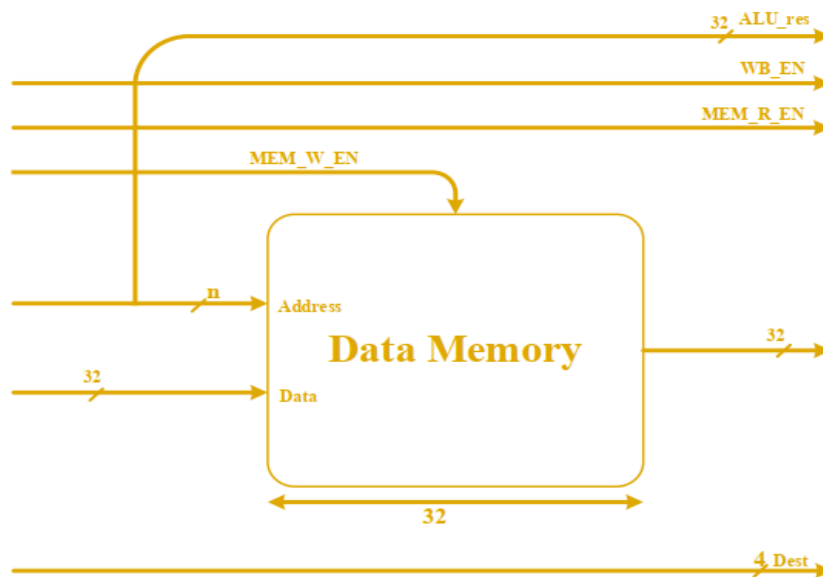
شکل ۱۱. نتیجه شبیه‌سازی EXE Stage



شکل ۱۲. نتیجه شبیه‌سازی EXE Stage با FPGA

MEM Stage .5

در این بخش عملیات‌هایی که نیاز به دسترسی به حافظه دارد مانند خواندن یا نوشتن داده‌ها در حافظه، اجرا می‌شود. در اینجا اگر دسترسی نیاز به دسترسی به حافظه داشته باشد، اطلاعات مربوط به آدرس و داده‌ها با استفاده از خانه حافظه‌ای که مدنظر است خوانده یا نوشته می‌شود. شمای کلی این بخش به صورت زیر است که بخش اصلی آن ماژول Data Memory می‌باشد:



شکل ۱۳. شمای کلی Mem Stage

Distributed Memory Generator

در این ماژول خواندن به صورت asynchronous و نوشتن به صورت synchronous انجام می‌شود به عبارت دیگر برای خواندن از حافظه کافی است در آدرس ورودی آدرس مدنظر قرار بگیرد و مقدار آن آدرس در خروجی قرار خواهد گرفت اما برای نوشتن نیاز به سیگنال مناسب و همچنین لبه بالارونده کلاک داریم. این ماژول را هم با استفاده از خود vivado و با عمق ۸۱۹۲ و عرض ۳۲ تشکیل می‌دهیم.

```
module MEM_Stage(clk ,rst, MEM_W_EN , ALU_res , Data , MEM_OUT );
    input clk ,rst,MEM_W_EN;
    input[31:0] ALU_res , Data;
    output[31:0] MEM_OUT;

    dist_mem_gen_0 data_mem (
        .a(ALU_res[10:0]), // input wire [10 : 0] a
        .d(Data), // input wire [31 : 0] d
        .clk(clk), // input wire clk
        .we(MEM_W_EN), // input wire we
        .spo(MEM_OUT) // output wire [31 : 0] spo
    );
endmodule
```

WB Stage .6

این بخش تنها دارای یک MUX ۳۲ بیتی با دو ورودی می‌باشد که توسط MEM_R_EN خروجی آن مشخص خواهد شد. ورودی‌های این ماژول نتیجه ALU و خروجی حافظه هستند که اگر مقدار سیگنال سلکت یک باشد داده موجود در حافظه و در غیر این صورت خروجی ALU انتخاب می‌شود. این مقدار در RF نوشته خواهد شد. کد زیر کار این بخش را انجام می‌دهد:

```
MUX2to1 MUX32 (  
  .inA(ALU_Result_MEM_Reg) ,  
  .inB(MEM_OUT_MEM_Reg) ,  
  .sel(MEM_R_EN_MEM_Reg) ,  
  .out(mux32_out_MEM)  
);
```

Hazard Unit .7

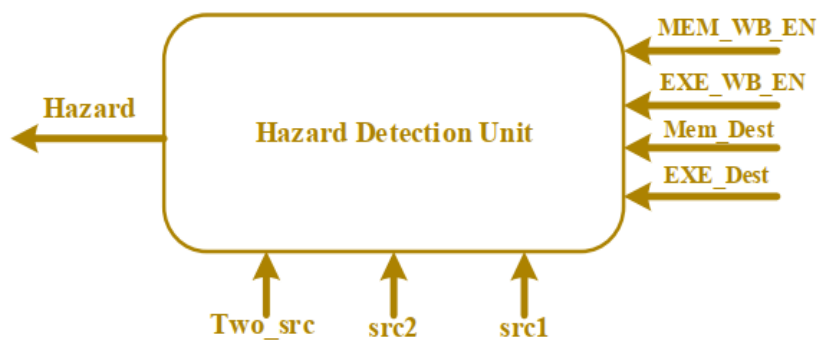
این واحد یکی از مهم‌ترین بخش‌های این پردازنده است که از اجرای نادرست دستورات در طراحی انجام شده جلوگیری می‌کند. سه نوع مخاطره داریم که به شرح زیر می‌باشند:

۱. مخاطره ساختاری: مخاطره ساختاری در معماری پردازنده به مخاطراتی اشاره دارد که ناشی از ساختار داخلی یک پردازنده است. به طور معمول در یک پردازنده عملیات‌های گوناگونی به صورت همزمان انجام می‌شوند که ممکن است موجب تداخل و خطا در اجرای برنامه ما شوند. درواقع اگر بخواهیم همزمان در Register File بنویسیم و هم بخوانیم این مشکل به وجود می‌آید. این مخاطره را با خواندن در لبه پایین رونده کلاک و نوشتن در لبه بالارونده آن در طراحی و کدنویسی پردازنده خود رفع کرده‌ایم.

۲. مخاطره کنترلی: در این طراحی چون تشخیص و محاسبه آدرس پرش تاخیر دارد و در EXE اتفاق می‌افتد ممکن است دو دستور بعدی به اشتباه وارد پایپ‌لاین شوند. به منظور رفع این مخاطره از سیگنال flush استفاده کرده‌ایم تا در زمان تشخیص پرش دو دستور نادرست وارد شده را از رجیسترهای دو مرحله قبل پاک کند.

۳. مخاطره داده‌ای: به طور کلی زمانی رخ می‌دهد که یک دستور نیاز به داده‌ای دارد که هنوز توسط دستور قبلی محاسبه یا در رجیستر ذخیره نشده‌است و خود سه حالت (RAW) Read after Write، (WAR) Write after Read و (WAW) Write after Write دارد که در این پردازنده ما با RAW مواجهیم. راه حل رفع این مخاطره این است که خط لوله تا زمانی که داده مورد نیاز آن آماده شود

متوقف می‌شود. درواقع مقادیر دو سورس وارد شده یعنی Rn و Rn را باید با رجیستر مقصد در بخش EXE مقایسه کنیم. اگر هر کدام از این دو با آن برابر بود باید توقف اتفاق بیفتد. همین حالت را در بخش حافظه نیز داریم و باید این دو را با مقصد حافظه نیز مقایسه کنیم و در صورت برابری نیاز به توقف داریم. همچنین باید توجه کرد که لازمه این موضوع این است که مقدار WB_EN نیز یک باشد.



شکل ۱۴. شمای کلی بخش تشخیص مخاطره

```
module Hazard (
    input clk,
    input [ 3 : 0 ] Src1,
    input [ 3 : 0 ] Src2,
    input two_src,
    input EXE_WB_EN,
    input [ 3 : 0 ] EXE_Dest,
    input MEM_WB_EN,
    input EN,
    input EXE_MEM_R_EN,
    input [ 3 : 0 ] MEM_Dest,
    output reg HAZARD
);

always @( * ) begin
    HAZARD = 1'b0 ;

    if (EXE_WB_EN == 1'b1) begin

        if (Src1 == EXE_Dest)
            HAZARD = 1'b1 ;

        if ((two_src == 1'b1 ) && (Src2 == EXE_Dest))
            HAZARD = 1'b1 ;
        end

    if (MEM_WB_EN == 1'b1) begin
        if (Src1 == MEM_Dest)
            HAZARD = 1'b1 ;

        if ((two_src == 1'b1 ) && (Src2 == MEM_Dest))
            HAZARD = 1'b1 ;
        end
    end

endmodule
```

```

module Hazard (
    input clk,
    input [3:0] Src1, Src2,
    input two_src,
    input EXE_WB_EN,
    input [3:0] EXE_Dest,
    input MEM_WB_EN,
    input EN,
    input EXE_MEM_R_EN,
    input [3:0] MEM_Dest,
    output reg HAZARD
);
always @(*) begin
    HAZARD = 1'b0;

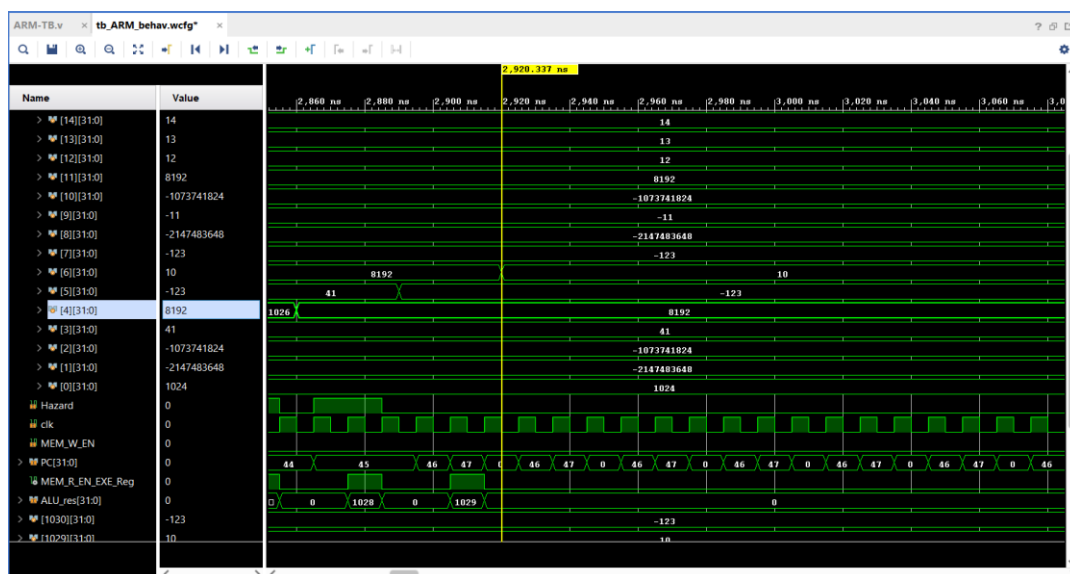
    if (EN) begin
        if (EXE_MEM_R_EN && EXE_WB_EN) begin
            if (Src1 == EXE_Dest) HAZARD = 1'b1;
            if (two_src && Src2 == EXE_Dest) HAZARD = 1'b1;
        end
    end

    else begin
        if (EXE_WB_EN && (Src1 == EXE_Dest || (two_src && Src2 == EXE_Dest)))
            HAZARD = 1'b1;
        if (MEM_WB_EN && (Src1 == MEM_Dest || (two_src && Src2 == MEM_Dest)))
            HAZARD = 1'b1;
    end
end
endmodule

```

کد اول کد این ماژول بدون فورواردینگ و حالت دوم کد به همراه فورواردینگ است.

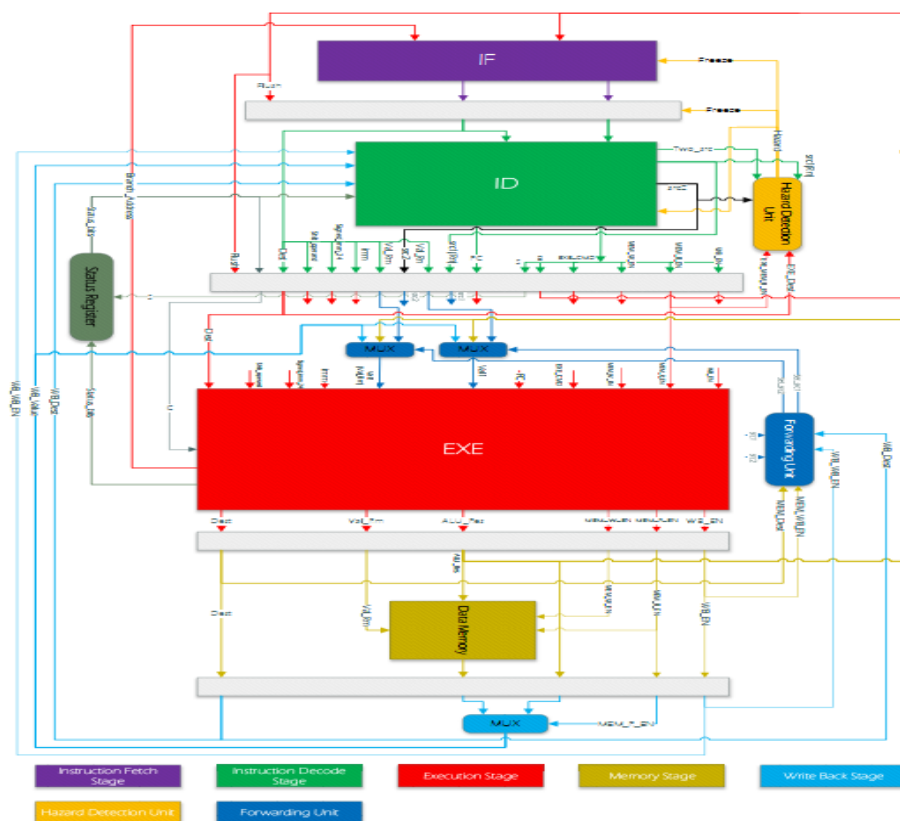
نتیجه شبیه‌سازی این بخش به صورت زیر است:



شکل ۱۵. نتیجه شبیه‌سازی بخش Hazard

8. Forwarding Unit

در بخش توضیح داده شد که مخاطره داده‌ای یکی از مشکلات کلی پردازنده‌ها و این پردازنده ARM طراحی شده در آزمایشگاه است. در هنگام رخ دادن این مخاطره پردازنده دچار توقف می‌شود که بازدهی ما را کاهش می‌دهد و زمان اجرای برنامه را افزایش خواهد داد. یکی از راه‌های اصلاح این مشکل استفاده از واحدی به نام Forwarding Unit است. این واحد با استفاده از چند MUX و منطق‌های کنترلی می‌تواند داده مورد نیاز برای دستور فعلی را قبل از اینکه در حافظه یا رجیستر نوشته شود از خروجی ALU دریافت کرده و مورد استفاده قرار دهد. این واحد با دریافت سیگنال‌های مرتبط با مراحل EXE, MEM و WB مشخص می‌کند که آیا داده مقصد دستور قبلی با رجیسترهای سورس دستور فعلی یکی است یا خیر. اگر یکسان باشند سیگنال انتخابی مناسب به MUX داده می‌شود تا داده صحیح مستقیم از مسیر Forwarding دریافت شود. اگر داده موردنظر از بخش MEM قابل دستیابی باشد سلکت به مقدار $2'b01$ درمی‌آید و اگر از مرحله WB باید دریافت شود سلکت به صورت $2'b10$ درمی‌آید. البته باید خاطر نشان کرد که این واحد نمی‌تواند مشکل دستور LDR را رفع کند. یعنی اگر داده‌ای در مرحله MEM باشد و هنوز از حافظه خوانده نشده باشد و دستور بعدی بلافاصله به همان داده نیاز داشته باشد، امکان فوروارد کردن وجود نخواهد داشت و همچنان باید پردازنده متوقف شود. همچنین واحد هازارد باید اندکی اصلاح می‌شد که کد آن در بخش قبل قرار گرفته شده است.



شکل ۱۶. نمای کلی ARM-Forwarding

```

module Forwarding_Unit (
    input en,
    input [3:0] src1, src2,
    input wb_en_mem,
    input [3:0] dest_mem,
    input wb_en_wb,
    input [3:0] dest_wb,
    output reg [1:0] sel_src1, sel_src2
);
always @(*) begin

    sel_src1 = 2'b00;
    sel_src2 = 2'b00;

    if (en) begin
        // Forwarding for Src1
        if (wb_en_mem && dest_mem == src1) sel_src1 = 2'b01; // From MEM
        else if (wb_en_wb && dest_wb == src1) sel_src1 = 2'b10; // From WB

        // Forwarding for Src2
        if (wb_en_mem && dest_mem == src2) sel_src2 = 2'b01; // From MEM
        else if (wb_en_wb && dest_wb == src2) sel_src2 = 2'b10; // From WB

    end

end
endmodule

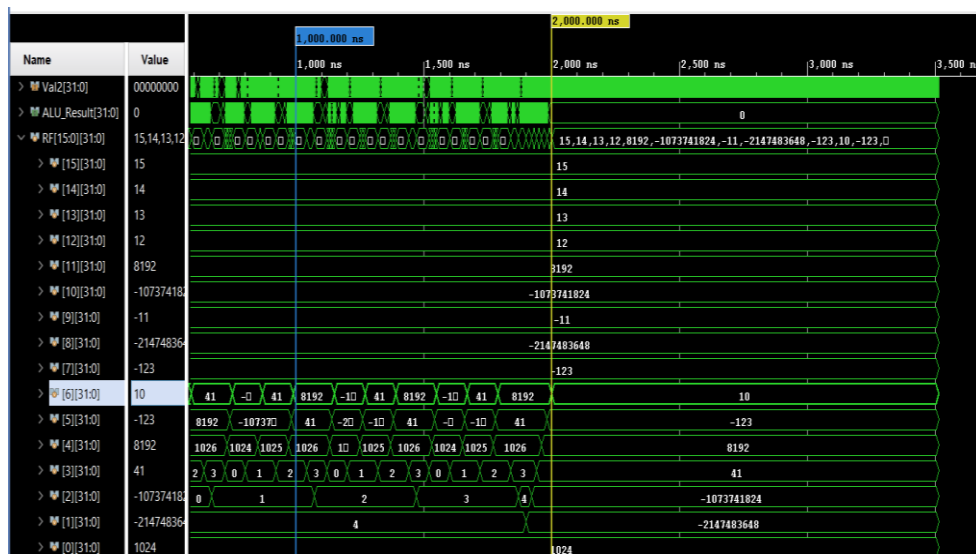
```

```

Mux3to1_32 M1(.a(Val_Rn_ID_reg) , .b(ALU_Result_EXE_Reg) , .c(mux32_out_MEM) , .sel(sel_src1) , .out(Val1_Forwarding));
Mux3to1_32 M2(.a(Val_Rm_ID_reg) , .b(ALU_Result_EXE_Reg) , .c(mux32_out_MEM) , .sel(sel_src2) , .out(Val2_Forwarding));

```

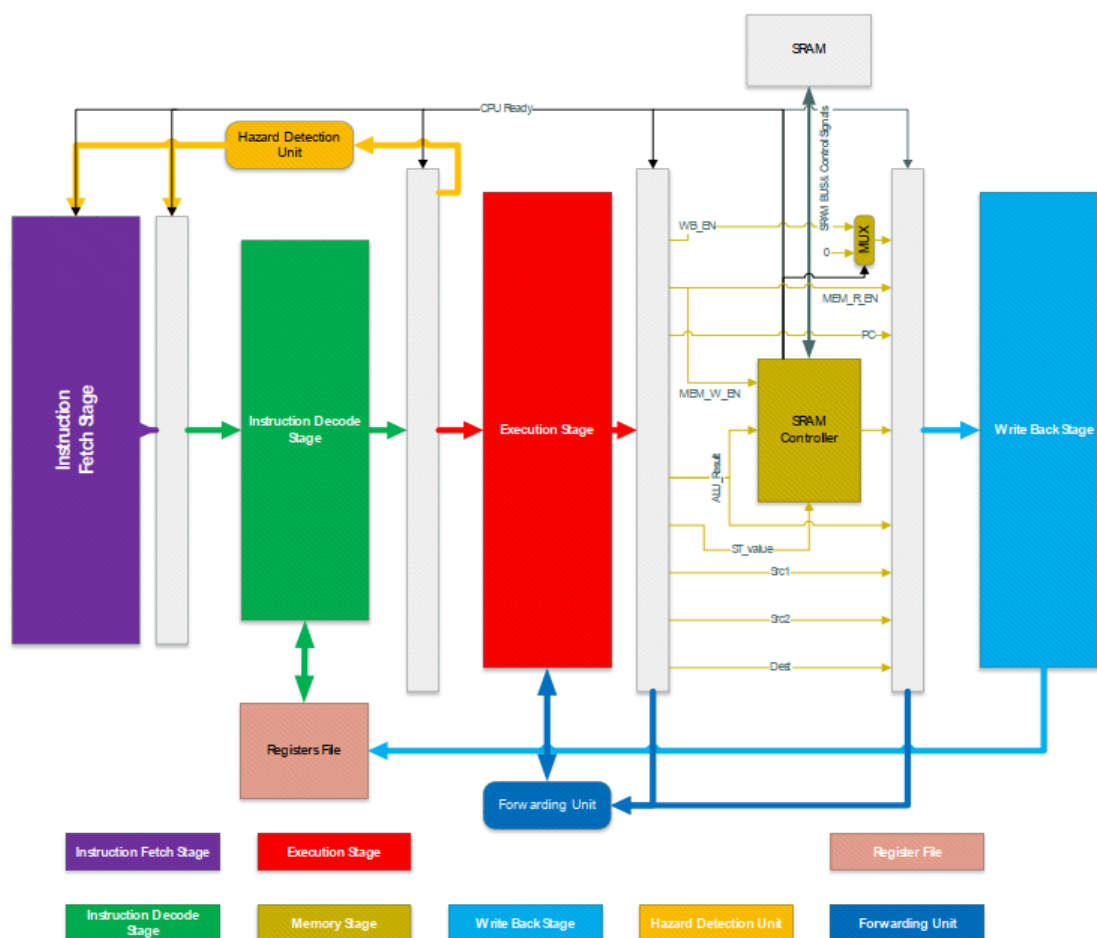
نتایج شبیه‌سازی در این بخش به صورت زیر است:



شکل ۱۷. نتیجه شبیه‌سازی ARM-Forwarding

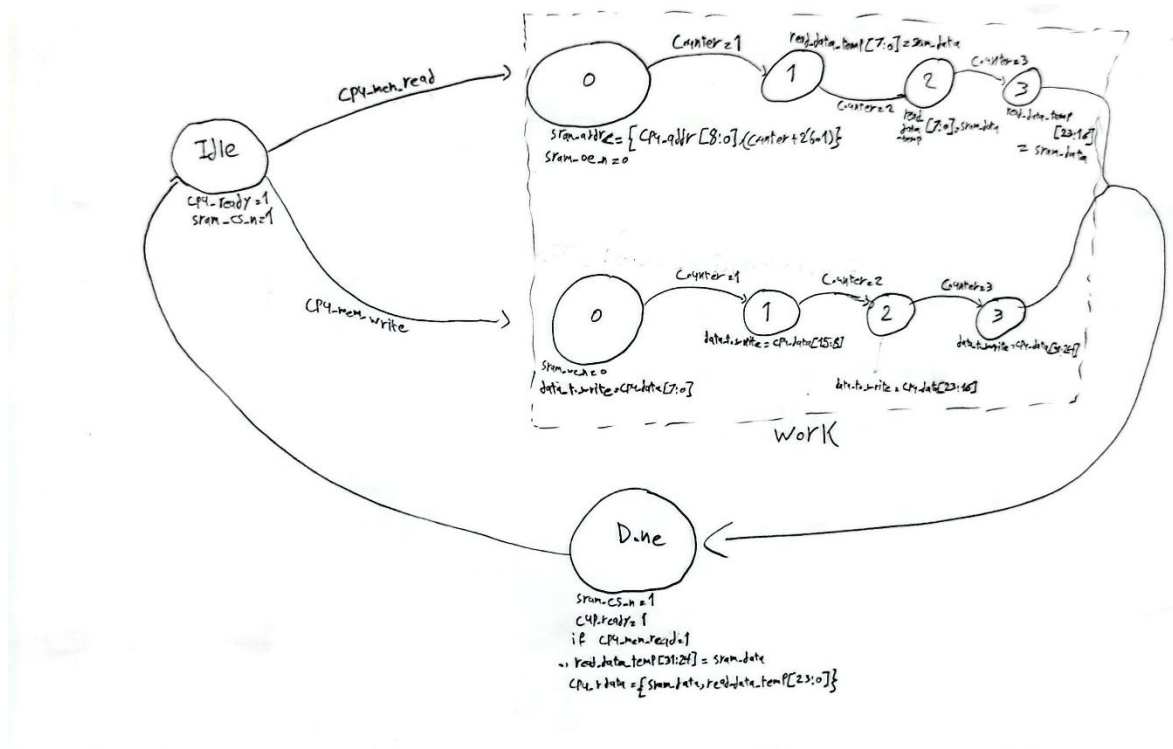
SRAM .9

در اینگونه پردازنده‌ها معمولاً حافظه مربوط به Instruction و data جزئی از پردازنده نیستند. همچنین حافظه موجود در تراشه FPGA معمولاً بسیار محدود است بنابراین می‌توان از حافظه‌های خارجی مانند SRAM و DRAM استفاده کرد. دیتا مموری ای که استفاده شد به صورت ایده‌آل یعنی تک سیکل و ۳۲ بیتی در نظر گرفته شده بود اما حافظه‌های خارجی ممکن است پهنای باند متفاوتی داشته باشند. در اینجا ماژول SRAM پهنای باند داده ۸ بیتی و ظرفیت دو کیلوبایت را داراست و یک سیکل تاخیر خواندن دارد. از آنجایی که پردازنده ARM ۳۲ بیتی است و در هر دستور LDR یا STR نیاز به خواندن یا نوشتن ۳۲ بیت داده دارد، ما نمی‌توانیم مستقیماً این پردازنده را به حافظه متصل کنیم و باید به وسیله یک کنترلر که بین پردازنده و حافظه قرار گرفته است عملیات ۳۲ بیتی را مدیریت کنیم. انتظار داریم که استفاده از این حافظه خارجی باعث افزایش زمان کل اجرای برنامه شود چرا که به جای یک سیکل خواندن هر داده ۳۲ بیتی باید در ۴ مرحله خوانده یا نوشته شود. شمای کلی این پردازنده با جایگزینی SRAM به فرم زیر است:



شکل ۱۸. ARM به همراه SRAM

همچنین استیت ماشین استفاده شده به شکل زیر است که البته برای سادگی به این شکل کشیده شده و در کد همانطور که مشخص ایت شامل سه استیت اصلی که در هر کدام تعدادی شرط وجود دارد می باشد.



شکل ۱۹. استیت ماشین SRAM

همچنین نتیجه شبیه سازی SRAM به صورت زیر است که مطابق انتظار زمان کل اجرای دستورات افزایش پیدا کرده است. همچنین چون نتیجه نهایی رجیسترها دقیقاً نتیجه مورد انتظار است پس سیگنال cpu_ready به درستی عمل کرده و توقف پردازنده نیز صحیح می باشد. همچنین نوشتن در حافظه به درستی در حال انجام است. (با این حال خروجی سیمولیشن ویوآدو به همراه فایل کامل در بخش SRAM قرار گرفته و اگر نیاز به بررسی بیشتری هست از آنجا می توان این کار را انجام داد).

کد کلی SRAM controller نیز در زیر آورده شده است:

```

module SRAM_Controller (
    input wire clk,
    input wire rst,

    input wire [31:0] cpu_addr,
    input wire [31:0] cpu_wdata,
    input wire cpu_mem_read,
    input wire cpu_mem_write,
    output reg [31:0] cpu_rdata,
    output reg cpu_ready,

    output reg [10:0] sram_addr,
    inout wire [7:0] sram_data,
    output reg sram_cs_n,
    output reg sram_oe_n,
    output reg sram_we_n
);

    localparam IDLE = 2'd0;
    localparam WORK = 2'd1;
    localparam DONE = 2'd2;

    reg [1:0] current_state, next_state;
    reg [31:0] read_data_temp;

    reg [1:0] counter;

    reg [7:0] data_to_write;
    assign sram_data = (!sram_we_n) ? data_to_write : 8'bz;

    always @(posedge clk, posedge rst) begin
        if (rst) begin
            current_state <= IDLE;
            counter <= 2'd0;
        end else begin
            current_state <= next_state;

            if (current_state == WORK) begin
                counter <= counter + 1;
            end else if (current_state == IDLE) begin
                counter <= 2'd0;
            end
        end
    end

    always @(*) begin
        next_state = current_state;
        case (current_state)
            IDLE: begin
                if (cpu_mem_read || cpu_mem_write)
                    next_state = WORK;
            end

            WORK: begin
                if (counter == 2'd3)
                    next_state = DONE;
            end
        end
    end

```

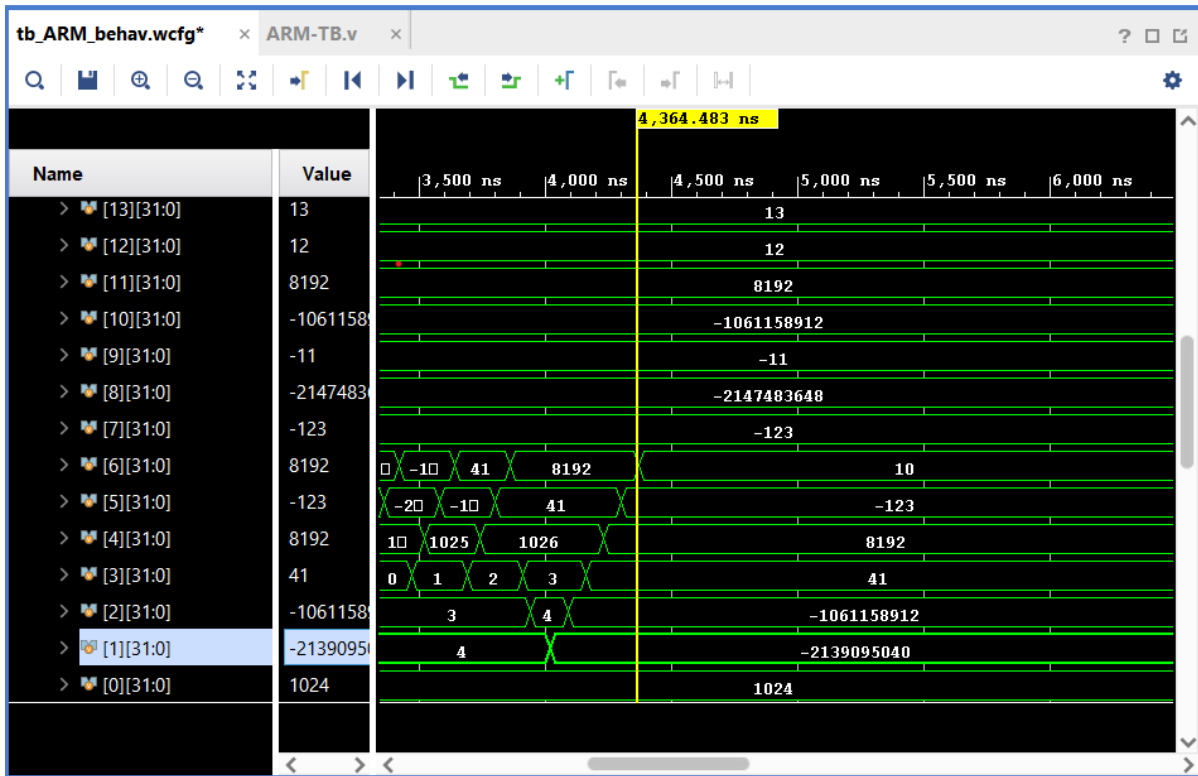
```

        DONE: begin
            next_state = IDLE;
        end
    endcase
end
always @(negedge clk, posedge rst) begin
    if (rst) begin
        cpu_ready <= 1'b1;
        sram_cs_n <= 1'b1;
        sram_oe_n <= 1'b1;
        sram_we_n <= 1'b1;
        cpu_rdata <= 32'd0;
        read_data_temp <= 32'd0;
        sram_addr <= 11'd0;
    end
    else begin
        sram_cs_n <= 1'b0;
        sram_oe_n <= 1'b1;
        sram_we_n <= 1'b1;
        cpu_ready <= 1'b0;

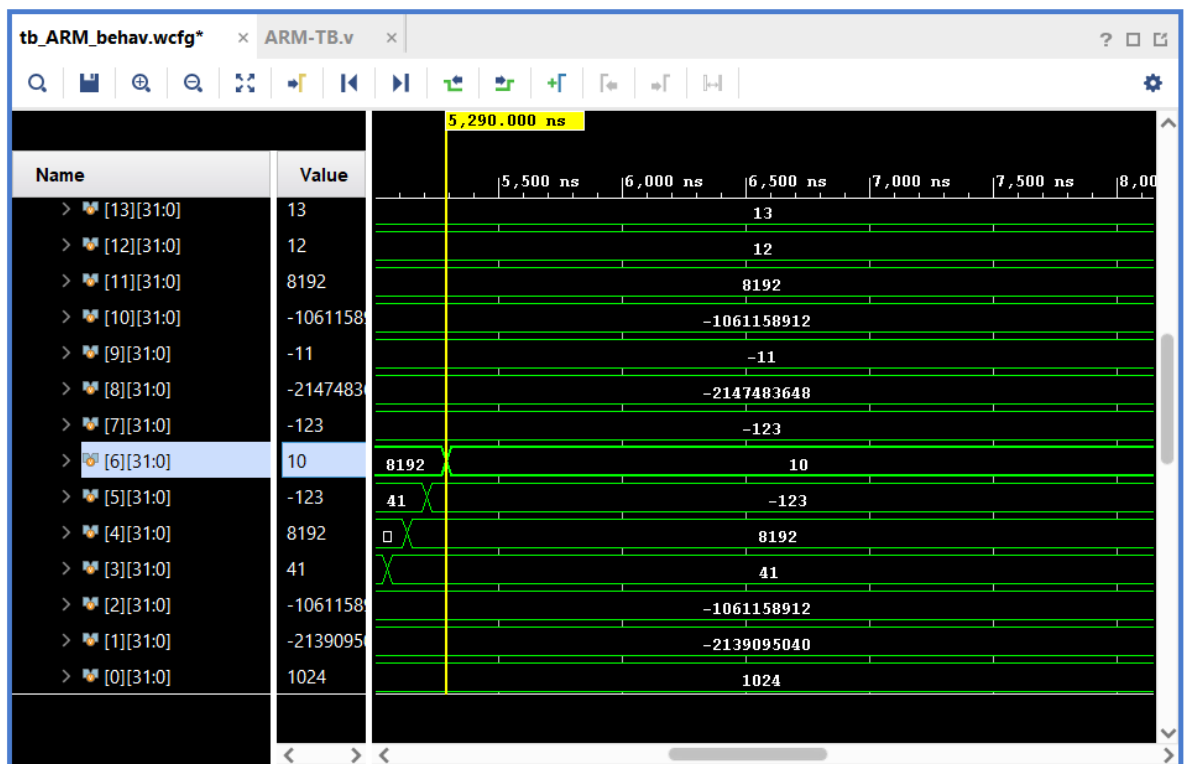
        case (current_state)
            IDLE: begin
                cpu_ready <= 1'b1;
                sram_cs_n <= 1'b1;
                if (cpu_mem_read || cpu_mem_write) begin
                    cpu_ready <= 1'b0;
                    sram_cs_n <= 1'b0;
                    sram_addr <= {cpu_addr[8:0], 2'b00};
                end
            end
            WORK: begin
                if (counter < 2'd3) begin
                    sram_addr <= {cpu_addr[8:0], (counter + 2'b01)};
                end

                if (cpu_mem_read) begin
                    sram_oe_n <= 1'b0;
                    if (counter == 2'd1) read_data_temp[7:0] <= sram_data;
                    if (counter == 2'd2) read_data_temp[15:8] <= sram_data;
                    if (counter == 2'd3) read_data_temp[23:16] <= sram_data;
                end
                if (cpu_mem_write) begin
                    sram_we_n <= 1'b0;
                    case (counter)
                        2'd0: data_to_write <= cpu_wdata[7:0];
                        2'd1: data_to_write <= cpu_wdata[15:8];
                        2'd2: data_to_write <= cpu_wdata[23:16];
                        2'd3: data_to_write <= cpu_wdata[31:24];
                    endcase
                end else begin
                    sram_oe_n <= 1'b0;
                end
            end
            DONE: begin
                sram_cs_n <= 1'b1;
                cpu_ready <= 1'b1;
                if (cpu_mem_read) begin
                    read_data_temp[31:24] <= sram_data;
                    cpu_rdata <= {sram_data, read_data_temp[23:0]};
                end
            end
        endcase
    end
end
endmodule

```



شکل ۲۰. شبیه‌سازی SRAM با Forwarding



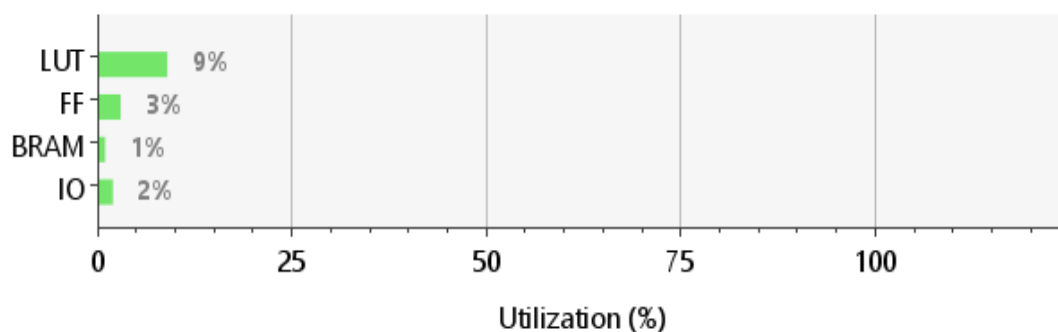
شکل ۲۱. شبیه‌سازی SRAM بدون Forwardin

10. گزارش عملکرد

حال خواسته‌های بخش پیاده‌سازی گزارشکار را به انجام می‌رسانیم: (با SRAM)

Resource	Utilization	Available	Utilization %
LUT	1567	17600	8.90
FF	1152	35200	3.27
BRAM	0.50	60	0.83
IO	2	100	2.00

جدول ۵. تعداد کل LUT ها



شکل ۲۲. Utilization

Name	Used
✓ N ARM_wrapper	1567
> I ARM_inst (ARM)	1458
> I VIO (vio_0)	102
I deb_inst (debouncer)	7

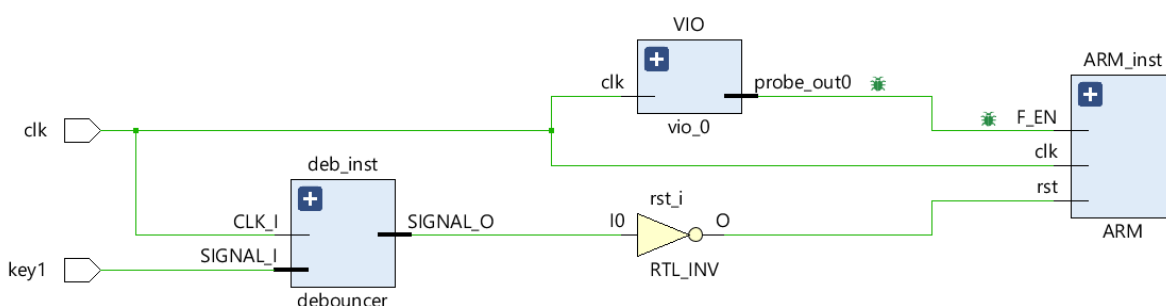
جدول ۶. LUT as Logic

Name	Used
✓ N ARM_wrapper	1152
> I ARM_inst (ARM)	904
> I VIO (vio_0)	231
I deb_inst (debouncer)	17

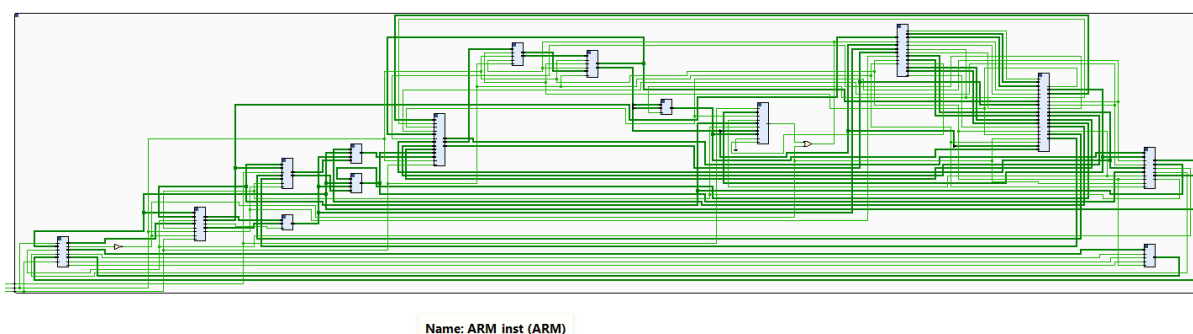
جدول ۷. Total FF

Name	Used
ARM_wrapper	0.500
ARM_inst (ARM)	0.500

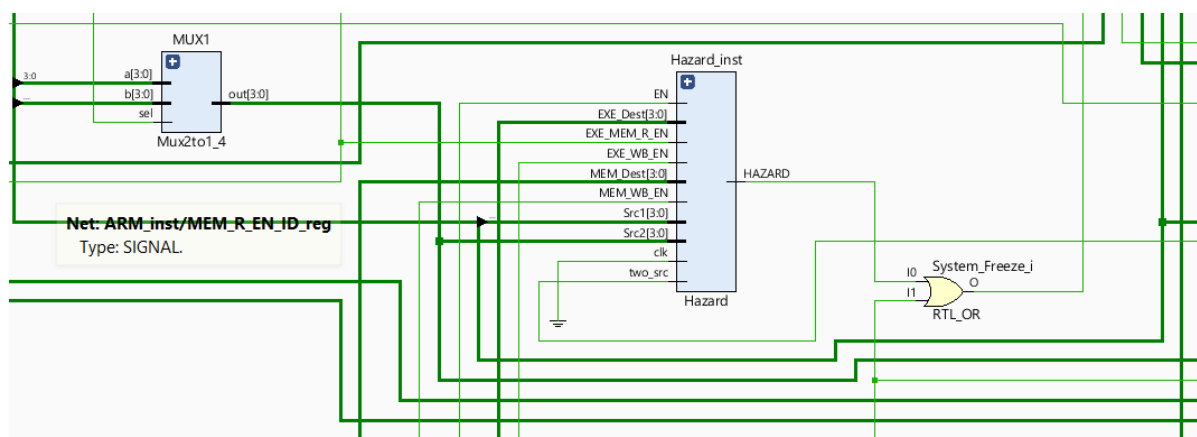
جدول ۸. Block RAM



شکل ۲۳. نمای کلی پیاده‌سازی Synthesized Design



شکل ۲۴. نمای کلی ARM سنتز شده



شکل ۲۵. زوم بیشتر روی یکی از بخش‌های ARM برای راستی آزمایی

زمان کل اجرای برنامه با مموری و بدون فورواردینگ:

$$2920 \text{ ns} = 292 \text{ clk} \rightarrow CPI = \frac{292}{47} = 6.213$$

زمان کل اجرای برنامه با مموری و با فورواردینگ:

$$2000 \text{ ns} = 200 \text{ clk} \rightarrow CPI = \frac{200}{47} = 4.255$$

$$\text{Speed Up} = \frac{6.213 - 4.255}{6.213} = 31.51\%$$

زمان کل اجرای برنامه با SRAM و بدون فورواردینگ:

$$5290 \text{ ns} = 529 \text{ clk} \rightarrow CPI = \frac{529}{47} = 11.255$$

زمان کل اجرای برنامه با SRAM و فورواردینگ:

$$4370 \text{ ns} = 437 \text{ clk} \rightarrow CPI = \frac{437}{47} = 9.2979$$

$$\text{Speed Up} = \frac{11.255 - 9.2979}{11.255} = 17.39\%$$

11. مشکلات هنگام انجام پروژه

سیم‌کشی‌ها: می‌توان گفت مهم‌ترین مشکلی که در طول این آزمایشگاه به آن برخورد کردیم مشکلات مربوط به سیم‌کشی و اتصالات بود. از ایرادات تاییپی گرفته تا بعضاً متصل کردن جابجای برخی سیگنال‌ها به یکدیگر. این مشکلات باعث z یا x شدن مقدار برخی از سیگنال‌های مهم می‌شد که کار را خراب می‌کرد.

اینستنس‌گیری اشتباه: مورد دیگری که گاهی باعث شد تا حدود یک جلسه از وقت ما گرفته شود اشتباه تاییپی در زمان اینستنس گرفتن از ماژول‌ها بود که با وجود اینکه این اشتباه تاییپی اسمی را به وجود می‌آورد که اصلاً وجود نداشت برنامه بدون ارور دادن در vivado ران می‌شد و به نتیجه غلط می‌رسید که تقریباً به صورت اتفاقی متوجه این مورد شدیم و آن را برطرف کردیم.

مخاطره ساختاری: از دیگری مشکلاتی که به آن برخوردیم این بود که طبق عادت همه سیگنال‌ها را به posedge حساس می‌کردیم که به خاطر مسئله مخاطره داده‌ای و نوشتن و خواندن در دوبله مختلف کلاک این باعث بروز مشکلاتی هم در Register File و هم در اعمال بخش SRAM شد که با درست کردن آن و توجه به لبه مناسب نتایج درست شدند.

خطاهای سنتز: در زمان سنتز در vivado گاهی generate bitstream دچار خطا می‌شد که این خطا به سادگی و با کارهایی مانند باز و بسته کردن برنامه برطرف می‌شد. همچنین پورت دستگاه FPGA پورت چندان مناسبی نبود و باید حواس خود را جمع می‌کردیم تا خیلی دچار تغییر مکان نشود.